

TECHNICAL ARTICLE & OPERATIONS GUIDE

Deep Dive into PostgreSQL

*Internal Architecture, Processes, Memory, WAL, and Operational
Diagnostics*

Validated against PostgreSQL 18.6 official documentation

Technical adaptation prepared September 2026

Original article credit “Deep Dive into PostgreSQL: Unveiling the Internal Architecture” by Arya Pathak, published July 14, 2024 on LinkedIn. This document expands, restructures, and technically validates the topic; it does not claim ownership of the original article or supplied artwork.

Document Control

Field	Value
Document type	Technical Article and Operations Guide
Production baseline	PostgreSQL 18.6
Original article author	Arya Pathak
Original publication	LinkedIn — July 14, 2024
Validation authority	PostgreSQL Global Development Group official documentation
Scope	Single-instance processes, memory, WAL, recovery, replication, MVCC, monitoring, and troubleshooting

Source and Accuracy Statement

The source article supplies the topic sequence and illustrations. Technical claims, settings, process roles, SQL, and version behavior were checked against PostgreSQL 18 documentation. Where the article is simplified or inaccurate, the corrected PostgreSQL 18 behavior is used and documented in Section 14.

Image credit The supplied overview image contains the watermark backend.husseinnasser.com. The four supplied figures are reproduced for explanation in the positions matching the source article. Credit: original image/watermark owner as shown; supplied through the Arya Pathak LinkedIn article. No ownership is transferred or implied.

Table of Contents / Index

1. Executive Summary	4
2. PostgreSQL Instance Architecture	5
3. Main Server and Client Backends	6
4. Memory Architecture	7
5. Query and Transaction Flow	8
6. Auxiliary and Background Processes	9
7. Autovacuum Architecture	11
8. WAL, Checkpoints, and Crash Recovery	12
9. Replication and WAL Processes	13
10. Storage and MVCC Internals	14
11. Monitoring and Diagnostic Scripts	14
12. Troubleshooting Scenarios	16
13. Configuration and Production Guidance	17
14. Corrections to the Source Article	18
15. Production Checklist	19
16. Credits and Official References	19

1. Executive Summary

A PostgreSQL instance is a group of cooperating operating-system processes, shared memory, process-local memory, and database files. The main postgres server process supervises the instance and accepts client connections. Each normal client session receives a separate backend process. Auxiliary processes handle checkpoints, WAL writing, buffer cleaning, logging, autovacuum coordination, recovery, archiving, replication, and other specialized work.

Reliability depends on Write-Ahead Logging: a data-page change must be described in WAL and WAL must reach durable storage before the corresponding dirty data page is allowed to be written safely. After a crash, recovery starts from a checkpoint redo location and replays WAL forward to restore consistent data files.

Architecture principle Backend isolation does not mean unlimited connections are safe. Every connection consumes process slots and memory. Use measured max_connections, application connection pooling, transaction discipline, and workload-specific memory limits.

2. PostgreSQL Instance Architecture

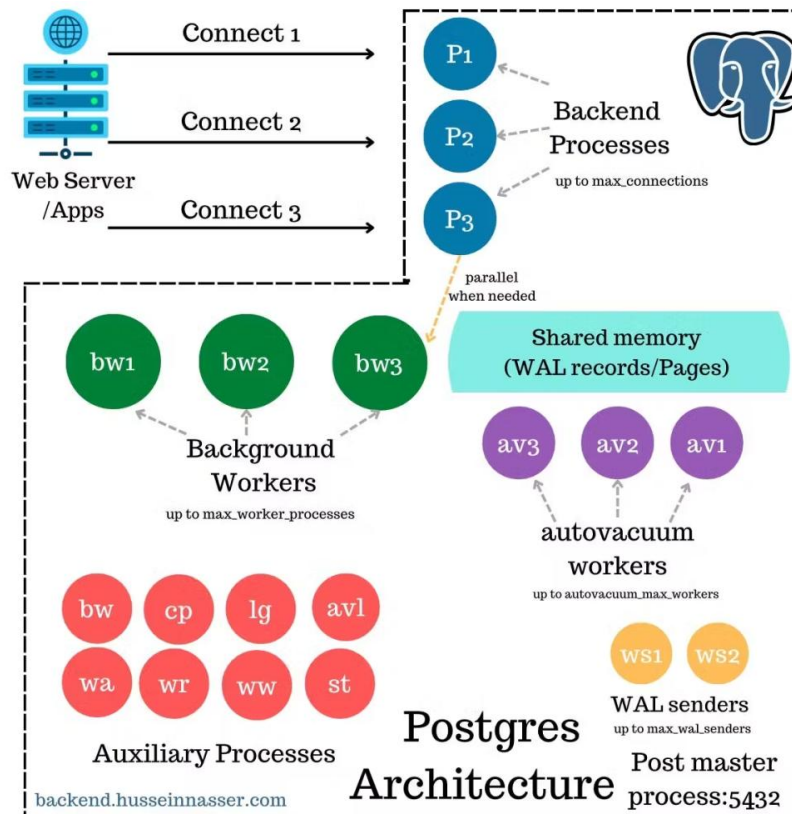


Figure 1 — Supplied PostgreSQL process overview. Credit: watermark owner backend.husseinnasser.com; reproduced from the Arya Pathak LinkedIn article.

Layer	Main components	Responsibility
Client/front end	Applications, psql, drivers, poolers	Open sessions, send SQL, receive results.
Supervisor	Main postgres server process (often called postmaster)	Initialize instance, listen, authenticate/launch backends, supervise children.
Session execution	Client backend processes	Parse, plan, execute, transact, and communicate.
Shared memory	shared_buffers, WAL buffers, locks, process state	Coordinate processes and cache shared database/WAL state.
Auxiliary processes	checkpointer, bgwriter, WAL writer, logger, startup, archiver, receiver, summarizer	Perform instance-wide specialized tasks.
Workers	parallel, logical replication, autovacuum, extensions	Perform bounded background or parallel work.
Persistent storage	data files, indexes, pg_wal, control/status files	Store durable database state and recovery information.

The diagram is conceptual, not a complete PostgreSQL 18 process inventory. Actual processes depend on configuration, activity, recovery/replication role, extensions, and operating system.

3. Main Server and Client Backends

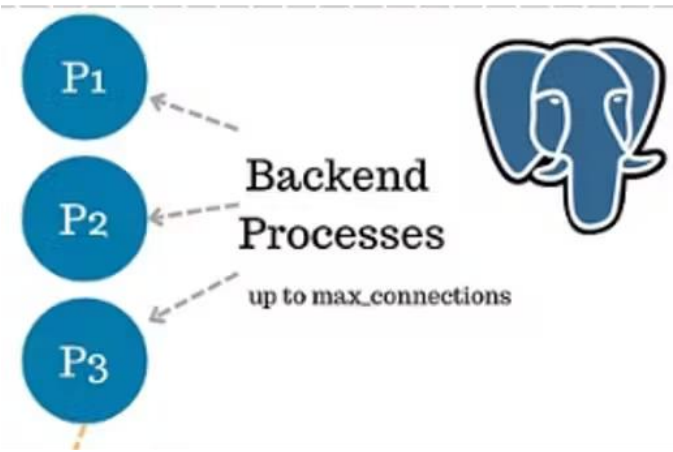


Figure 2 — Process-per-connection model. Credit: supplied through the Arya Pathak article; derived from Figure 1.

The executable and server process are named postgres. “Postmaster” remains a widely used historical term for the supervisor process. It listens on configured TCP/IP addresses and Unix-domain sockets, accepts connection requests, and starts a separate server process for each connection. After startup, the client communicates directly with its backend without the supervisor handling each query.

Component	What it does	Operational risk
Main server process	Creates shared memory; launches/supervises child processes; accepts connections.	Failure or unsafe termination affects the instance.
Client backend	Authenticates session, runs SQL and transactions, uses shared/local memory.	Idle-in-transaction sessions retain snapshots/locks.
max_connections	Caps concurrent client connection slots.	Higher value increases reserved resources and possible memory demand.
Reserved connections	Keep access for administrative recovery according to configured reservation settings.	Poor sizing can lock administrators out during saturation.
Connection pooler	Reuses a smaller backend population for many application clients.	Transaction/session features must match pooling mode.

```

SELECT pid, backend_type, username, datname, state,
       backend_start, xact_start, wait_event_type, wait_event,
       left(query, 160) AS query
FROM pg_stat_activity
ORDER BY backend_type, backend_start;

SHOW port;
SHOW listen_addresses;
SHOW max_connections;
SHOW superuser_reserved_connections;

```

4. Memory Architecture

Memory area	Scope	Purpose / control
shared_buffers	Instance-wide shared memory	Caches table/index pages; server-start setting.
WAL buffers	Instance-wide shared memory	Holds WAL data not yet written; wal_buffers.
Lock and process structures	Instance-wide	Coordinates locks, process state, snapshots, and IPC.
work_mem	Per operation, potentially multiple per query/process	Sorts, hashes, and related executor work before spilling.
maintenance_work_mem	Per maintenance operation	VACUUM, CREATE INDEX, ALTER TABLE and similar work; exceptions apply.
autovacuum_work_mem	Per autovacuum worker	Overrides maintenance_work_mem for workers when configured.
temp_buffers	Per session	Temporary-table buffers, allocated as used.
Dynamic shared memory	Per parallel/background operation	Coordinates parallel workers and selected extensions.

Memory sizing warning `work_mem` is not a per-server budget. One query can have several memory-consuming nodes, multiple sessions run concurrently, parallel workers may multiply use, and hash operations can exceed the base through `hash_mem_multiplier`.

```
SELECT name, setting, unit, context, source
FROM pg_settings
WHERE name IN ('shared_buffers','wal_buffers','work_mem',
'maintenance_work_mem','autovacuum_work_mem','temp_buffers',
'max_connections','max_worker_processes',
'max_parallel_workers','max_parallel_workers_per_gather')
ORDER BY name;
```

5. Query and Transaction Flow

1. Client opens a connection; PostgreSQL authenticates using `pg_hba.conf` and the selected authentication method.
2. A backend process handles the session.
3. Parser analyzes SQL syntax and builds a parse tree.
4. Rewriter applies rules and expands views.
5. Planner/optimizer estimates alternatives using statistics and costs.
6. Executor runs the selected plan, reading shared buffers or requesting storage I/O.
7. Changes create new tuple versions under MVCC and generate WAL records.
8. COMMIT durability follows synchronous_commit and WAL-flush rules; locks and transaction state are released appropriately.
9. Dirty data pages can be written later because WAL provides redo protection.

```
EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS, VERBOSE)
SELECT * FROM public.orders WHERE customer_id = 1001;

SELECT locktype, mode, granted, relation::regclass, pid
FROM pg_locks
WHERE pid = pg_backend_pid();
```

MVCC allows readers and writers to coexist by using tuple visibility rules and snapshots. It does not eliminate blocking: row locks, heavyweight locks, predicate locks, buffer/content locks, and extension/application coordination can still matter.

6. Auxiliary and Background Processes

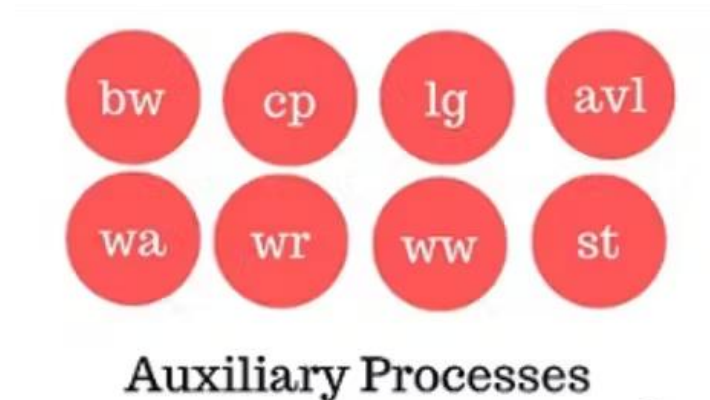


Figure 3 — Auxiliary-process abbreviations from the supplied diagram. Credit: supplied through the Arya Pathak article; derived from Figure 1.

Process	PostgreSQL 18 responsibility
checkpointer	Performs checkpoints and spreads dirty-buffer writes across the checkpoint interval.
background writer	Writes selected dirty shared buffers so clean reusable buffers are available.
WAL writer	Writes WAL buffers periodically; commit backends can also flush WAL.
logger	Collects stderr output and writes configured log files when logging collector is enabled.
startup process	Performs crash/archive/standby recovery by replaying WAL when recovery is required.
WAL archiver	Archives completed WAL segments when archive mode and command/library are configured.
WAL receiver	Receives streaming WAL on a standby.
WAL summarizer	Creates WAL summaries when summarize_wal is enabled.
autovacuum launcher	Schedules autovacuum workers across databases.
logical replication launcher	Coordinates logical replication workers when configured.

Checkpoint correction The checkpointer flushes dirty data pages and writes a checkpoint record, but WAL change records were already written/flushed according to WAL rules. PostgreSQL also has a separate WAL writer; do not combine these roles.

```
SELECT pid, backend_type, wait_event_type, wait_event
FROM pg_stat_activity
WHERE backend_type <> 'client backend'
ORDER BY backend_type, pid;

SELECT checkpoints_timed, checkpoints_req, checkpoint_write_time,
       checkpoint_sync_time, buffers_checkpoint, buffers_clean,
       maxwritten_clean, buffers_backend
FROM pg_stat_bgwriter;
```

7. Autovacuum Architecture

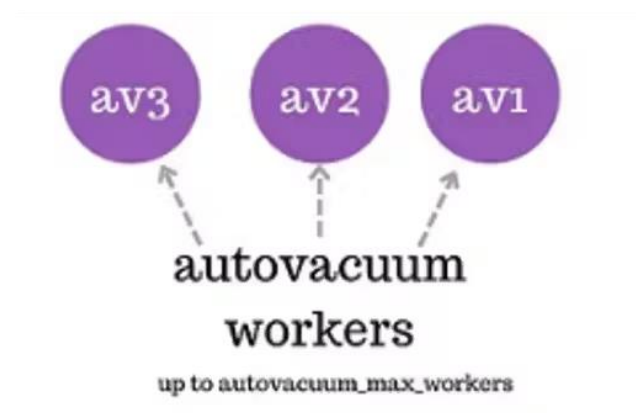


Figure 4 — Autovacuum workers. Credit: supplied through the Arya Pathak article; derived from Figure 1.

Autovacuum consists of one launcher and multiple workers. The launcher schedules workers among databases; each worker checks tables and runs VACUUM and/or ANALYZE when thresholds require it. PostgreSQL 18 separates the number of worker slots from the maximum workers that may run concurrently.

Function	Why it matters
Remove dead tuple/index references when safe	Makes internal space reusable and controls bloat.
Maintain visibility map	Supports page skipping and index-only scans.

Function	Why it matters
Freeze old transaction metadata	Prevents transaction ID and MultiXact wraparound.
ANALYZE changed tables	Refreshes planner statistics.
Insert-driven vacuum	Maintains visibility/freezing on append-heavy tables.

```

SELECT schemaname, relname, n_live_tup, n_dead_tup,
       last_autovacuum, last_autoanalyze,
       autovacuum_count, autoanalyze_count
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC LIMIT 30;

SELECT pid, datname, relid::regclass AS relation, phase,
       heap_blks_scanned, heap_blks_total, index_vacuum_count
FROM pg_stat_progress_vacuum;

SHOW autovacuum_max_workers;
SHOW autovacuum_worker_slots;

```

Space correction Standard VACUUM normally makes space reusable inside a relation; it does not generally shrink the file. VACUUM FULL rewrites and requires ACCESS EXCLUSIVE. Dead tuples can remain necessary while old snapshots, prepared transactions, or replication slots retain the horizon.

8. WAL, Checkpoints, and Crash Recovery

WAL protects data integrity by requiring the log record for a change to reach durable storage before the changed data page is written. A checkpoint guarantees that data files reflect WAL through a defined point and records recovery metadata. On crash, PostgreSQL reads control/checkpoint information and replays WAL forward from the redo location.

Stage	Action	Durability meaning
Modify	Backend changes a shared buffer and creates WAL records.	Data page can remain dirty in memory.
Commit	Required WAL is flushed according to commit configuration.	Committed change can be recovered even if page is not written.

Stage	Action	Durability meaning
Background write	bgwriter/backend/checkpointer may write dirty pages.	WAL-before-data rule must be satisfied.
Checkpoint	Dirty pages are flushed and checkpoint record/control data updated.	Recovery redo start can advance.
Crash recovery	Startup process replays WAL from redo location.	Restores data-file consistency; no normal SQL undo pass is required.

Recovery: The startup process is a child process used when recovery is needed; it is not a separate process that runs before the main postgres server exists. PostgreSQL crash recovery primarily redoes WAL. MVCC visibility makes incomplete transactions invisible; saying they are simply “rolled back by WAL replay” is misleading.

```
SELECT pg_current_wal_lsn(), pg_current_wal_insert_lsn();
SELECT * FROM pg_stat_wal;
SELECT * FROM pg_stat_checkpointer;
SHOW checkpoint_timeout;
SHOW max_wal_size;
SHOW full_page_writes;
SHOW synchronous_commit;
```

9. Replication and WAL Processes

Process	Runs where	Role
WAL sender	Primary or upstream	Streams physical WAL or logical-decoding output to clients/subscribers.
WAL receiver	Physical standby	Receives WAL stream and writes it for replay.
Startup process	Standby/recovery server	Replays received/restored WAL; supports hot standby when enabled.
Logical replication launcher/workers	Subscriber	Launches apply/synchronization workers and applies changes.

Process	Runs where	Role
Archiver	Archiving server	Copies completed WAL segments using archive_command/library.

```
SELECT pid, application_name, client_addr, state,
       sent_lsn, write_lsn, flush_lsn, replay_lsn,
       sync_state, reply_time
FROM pg_stat_replication;

SELECT status, receive_start_lsn, written_lsn, flushed_lsn,
       latest_end_lsn, latest_end_time
FROM pg_stat_wal_receiver;

SELECT slot_name, slot_type, active, restart_lsn,
       confirmed_flush_lsn, wal_status, safe_wal_size
FROM pg_replication_slots;
```

WAL archiving enables continuous archiving and point-in-time recovery when combined with a valid base backup and tested restore procedure. Streaming replication is not a backup by itself; corruption or unwanted changes can replicate.

10. Storage and MVCC Internals

Structure	Purpose
Heap relation	Stores table tuple versions in pages, normally 8 KB blocks.
Index relation	Stores access-method-specific keys/references; separate from heap.
TOAST relation	Stores/compresses oversized attribute values when required.
Free Space Map	Tracks pages with usable free space.
Visibility Map	Tracks all-visible and all-frozen heap pages.
pg_wal	Stores WAL segment files for recovery/replication.
pg_control	Stores critical cluster control/checkpoint state.
Tablespaces	Map database objects to configured filesystem locations.

UPDATE normally creates a new tuple version and marks the old version obsolete. DELETE marks a version as deleted. VACUUM removes versions only after they are no longer visible to any relevant snapshot. HOT updates can avoid new index entries when indexed columns are unchanged and suitable page space exists.

```
SELECT c.oid::regclass AS relation,
       pg_size_pretty(pg_relation_size(c.oid)) AS main_fork,
       pg_size_pretty(pg_indexes_size(c.oid)) AS indexes,
       pg_size_pretty(pg_total_relation_size(c.oid)) AS total
FROM pg_class c
WHERE c.relkind IN ('r','m')
ORDER BY pg_total_relation_size(c.oid) DESC LIMIT 30;

SELECT xmin, xmax, ctid, tableoid::regclass, *
FROM public.orders LIMIT 10;
```

11. Monitoring and Diagnostic Scripts

11.1 Process inventory and connection pressure

```
SELECT backend_type, state, count(*)
FROM pg_stat_activity
GROUP BY backend_type, state ORDER BY backend_type, state;

SELECT count(*) FILTER (WHERE backend_type='client backend') AS clients,
       current_setting('max_connections')::int AS max_connections
FROM pg_stat_activity;
```

11.2 Long transactions, blockers, and waits

```
SELECT pid, username, state, xact_start,
       now()-xact_start AS xact_age,
       wait_event_type, wait_event, left(query,160) AS query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY xact_start;

SELECT blocked.pid AS blocked_pid, blocker.pid AS blocker_pid,
       blocked.query AS blocked_query, blocker.query AS blocker_query
FROM pg_stat_activity blocked
CROSS JOIN LATERAL unnest(pg_blocking_pids(blocked.pid)) bpid
JOIN pg_stat_activity blocker ON blocker.pid=bpid;
```

11.3 WAL, checkpoints, and I/O

```
SELECT * FROM pg_stat_wal;
SELECT * FROM pg_stat_checkpointer;
SELECT backend_type, object, context, reads, writes,
       read_time, write_time, fsyncs, fsync_time
FROM pg_stat_io
ORDER BY backend_type, object, context;
```

11.4 Memory-relevant plan inspection

```
EXPLAIN (ANALYZE, BUFFERS, WAL, SETTINGS, SUMMARY)
SELECT customer_id, count(*)
FROM public.orders GROUP BY customer_id ORDER BY count(*) DESC;

SELECT pid, datname, temp_files, temp_bytes
FROM pg_stat_database d
JOIN pg_stat_activity a USING (datid)
WHERE a.backend_type='client backend';
```

Version note View columns change across major versions. These scripts target PostgreSQL 18.6. Verify them against the exact deployed minor release and managed-service permissions before automation.

12. Troubleshooting Scenarios

Symptom	Architecture checks	Safe direction
Too many connections	pg_stat_activity counts/states; pool; max_connections; reserved slots.	Fix leaks/idle sessions; pool; size from memory and workload—not only raise max.
High memory/OOM	Concurrent backends, parallelism, work_mem nodes, autovacuum/maintenance.	Reduce concurrency/per-operation memory; add pooling; inspect OS logs.
Checkpoint I/O spikes	pg_stat_checkpointer, pg_stat_io, WAL rate, max_wal_size, timeout.	Spread checkpoints; review WAL/storage; preserve recovery goals.

Symptom	Architecture checks	Safe direction
WAL growth	Slots, archiver failures, replication lag, wal_keep_size, backups.	Restore consumer/archive; remove only confirmed abandoned slot; add capacity.
Autovacuum lag	Progress, dead tuples, worker slots, locks, old snapshots.	Remove confirmed blocker; per-table tune; verify I/O/worker capacity.
Replica lag	Sender/receiver/apply LSNs, network, I/O, conflicts.	Find stage of lag; tune/capacity; protect retention.
Slow query with CPU/I/O	Plan, buffers, WAL, temp spill, waits, statistics.	Correct SQL/index/stats/memory after evidence.
Server will not start	Logs, postmaster.pid validity, port, permissions, disk, recovery files.	Do not delete state blindly; resolve exact error and protect data.
Crash recovery slow	WAL since checkpoint, storage latency, archive/restore speed.	Allow recovery; monitor logs/LSN; tune future checkpoint/archive design.
Index-only scan fetches heap	Visibility map and vacuum health.	Restore vacuum cadence; do not force plan.

Do not terminate blindly Backend termination rolls back the active transaction and can affect applications. Slot deletion can discard required WAL. CHECKPOINT can create I/O pressure. VACUUM FULL blocks with ACCESS EXCLUSIVE. Confirm ownership, impact, rollback, and recovery path first.

13. Configuration and Production Guidance

Area	Guidance
Connections	Pool application connections; reserve administration access; set idle/transaction timeouts only with application approval.
Memory	Budget shared plus worst-case local memory across sessions/workers; test parallel workloads.

Area	Guidance
WAL/checkpoints	Monitor WAL rate, checkpoint cause/duration, storage latency, and archive health.
Autovacuum	Use per-table thresholds for large/hot objects; monitor XID and MultiXact age.
Logging	Enable useful logging and rotation; protect sensitive SQL/data; centralize and retain appropriately.
Replication	Monitor LSN lag, slot retention, network, standby conflicts, and replay capacity.
Storage	Maintain disk headroom for pg_wal, maintenance, temp files, backups, and growth.
Security	Use least privilege, SCRAM/TLS as required, restrictive pg_hba.conf, and protected filesystem ownership.
HA/DR	Define RPO/RTO; combine replication with tested backups, WAL archives, and restore/failover exercises.
Extensions/workers	Review C background-worker extensions for trust, compatibility, resource slots, and crash impact.

```
SELECT name, setting, unit, context, source, pending_restart
FROM pg_settings
WHERE name IN ('max_connections','shared_buffers','work_mem',
'maintenance_work_mem','max_worker_processes','max_wal_senders',
'checkpoint_timeout','max_wal_size','autovacuum_max_workers',
'autovacuum_worker_slots','logging_collector','archive_mode')
ORDER BY name;
```

14. Corrections to the Source Article

Source statement or implication	Validated PostgreSQL 18 correction
Background workers were introduced in PostgreSQL 9.6.	The background worker infrastructure predates 9.6; parallel query arrived in 9.6. Do not equate the two.

Source statement or implication	Validated PostgreSQL 18 correction
max_worker_processes alone controls all workers.	It is an overall limit, but parallel workers, autovacuum workers/slots, WAL senders, and logical replication have additional limits/reservations.
Checkpoint writes all modified pages and WAL records.	Checkpoint flushes dirty data pages and writes checkpoint metadata/record; WAL records follow WAL-writer/backend flush rules.
Autovacuum reclaims storage space and prevents bloat.	It normally makes internal space reusable; it does not generally shrink relation files. It also freezes old tuple metadata and analyzes tables.
WAL archiver and receiver are one combined role.	Archiver copies completed WAL segments; receiver streams WAL on a standby. WAL senders run upstream.
Startup process runs before the postmaster.	The main postgres server starts and launches a startup child when recovery is required.
Crash recovery rolls back incomplete transactions.	Recovery redoes WAL to make files consistent; MVCC visibility leaves incomplete transactions invisible. There is no general SQL-style undo replay.
Backend count is limited only by max_connections.	Client slots relate to max_connections and reservation settings; auxiliary/workers use other slot limits and do not all count as clients.
Shared memory contains “WAL records/pages.”	Shared buffers hold database pages; WAL buffers hold WAL data. They are distinct shared-memory areas.
Processes communicate only through mutexes/semaphores.	PostgreSQL uses shared memory plus multiple synchronization/IPC mechanisms; the exact primitives are implementation details.
Fork and copy-on-write description applies everywhere.	Unix commonly uses fork; Windows uses a different process creation implementation while preserving process-per-session architecture.

15. Production Checklist

- ☐ Exact PostgreSQL 18.x version, operating system, packaging, and service constraints recorded.
- ☐ Process inventory and backend types captured during normal and peak workload.
- ☐ Connection count, pool behavior, reserved access, and transaction age monitored.
- ☐ Shared and local memory budget tested at realistic concurrency and parallelism.
- ☐ WAL generation, checkpoint behavior, archive status, and disk headroom alerted.
- ☐ Autovacuum progress, dead tuples, XID/MultiXact age, and blockers monitored.
- ☐ Replication sender/receiver/apply lag and slot WAL retention alerted.
- ☐ Backups, archived WAL, PITR, crash recovery, and failover tested—not assumed.
- ☐ `pg_hba.conf`, TLS, SCRAM, roles, filesystem ownership, and log exposure reviewed.
- ☐ Changes have evidence, nonproduction test, owner, rollback, and post-change validation.
- ☐ Diagrams are treated as conceptual; live process/view output is the operational truth.

16. Credits and Official References

16.1 Credits

Item	Credit
Original article	Arya Pathak — “Deep Dive into PostgreSQL: Unveiling the Internal Architecture,” LinkedIn, July 14, 2024.
Original article URL	https://www.linkedin.com/pulse/deep-dive-postgresql-unveiling-internal-architecture-arya-pathak-u7qsf/
Author profile	https://www.linkedin.com/in/arya-m-pathak/
Supplied architecture artwork	Watermark shown as backend.husseinasser.com; reproduced as supplied through the source article.
Technical adaptation	Prepared for Praveen Madupu; expanded and validated against PostgreSQL 18.6 official documentation.

16.2 PostgreSQL official references

- **Architectural Fundamentals:** <https://www.postgresql.org/docs/18/tutorial-arch.html>
- **Server Startup:** <https://www.postgresql.org/docs/18/server-start.html>

- **Kernel Resources and Shared Memory:** <https://www.postgresql.org/docs/18/kernel-resources.html>
- **Resource Consumption and Workers:** <https://www.postgresql.org/docs/18/runtime-config-resource.html>
- **WAL Settings:** <https://www.postgresql.org/docs/18/runtime-config-wal.html>
- **WAL Introduction:** <https://www.postgresql.org/docs/18/wal-intro.html>
- **WAL Configuration and Checkpoints:** <https://www.postgresql.org/docs/18/wal-configuration.html>
- **WAL Internals and Recovery:** <https://www.postgresql.org/docs/18/wal-internals.html>
- **Routine Vacuuming / Autovacuum:** <https://www.postgresql.org/docs/18/routine-vacuuming.html>
- **Monitoring Statistics:** <https://www.postgresql.org/docs/18/monitoring-stats.html>
- **Progress Reporting:** <https://www.postgresql.org/docs/18/progress-reporting.html>
- **Streaming Replication:** <https://www.postgresql.org/docs/18/warm-standby.html>
- **Logical Replication Architecture:** <https://www.postgresql.org/docs/18/logical-replication-architecture.html>
- **Database Physical Storage:** <https://www.postgresql.org/docs/18/storage.html>
- **Background Worker Processes:** <https://www.postgresql.org/docs/18/bgworker.html>
- **PostgreSQL Glossary — Auxiliary Processes:** <https://www.postgresql.org/docs/18/glossary.html>